

O que é uma API REST?

Uma **API REST** (Representational State Transfer) é um estilo de arquitetura usado para construir serviços web que permitem a comunicação entre sistemas de forma simples, padronizada e escalável — geralmente utilizando o protocolo HTTP.

É uma interface que permite que diferentes aplicações (frontend, mobile, outros sistemas) se comuniquem com um servidor por meio de **requisições HTTP**.

Ela segue os princípios definidos por Roy Fielding, que descreveu esse modelo em sua tese de doutorado.

Principais características

1. Uso de HTTP

As operações são feitas usando métodos HTTP:

Método	Ação	Exemplo
GET	Buscar dados	<code>/clientes</code>
POST	Criar	<code>/clientes</code>
PUT	Atualizar	<code>/clientes/1</code>
DELETE	Remover	<code>/clientes/1</code>

2. Recursos (Resources)

Tudo na API é tratado como um **recurso**, identificado por uma URL.

Exemplo:

`/clientes`
`/pedidos`
`/produtos/10`

3. Stateless (sem estado)

Cada requisição é independente. O servidor **não armazena estado do cliente** entre requisições. Toda requisição deve conter todas as informações necessárias.

4. Formato de dados

Normalmente utiliza:

- JSON (mais comum)
- XML (menos usado hoje)

Exemplo JSON:

```
{
  "id": 1,
  "nome": "João",
  "saldo": 1000.00
}
```

5. Interface uniforme

A API segue padrões bem definidos para facilitar o uso:

- URLs consistentes
- Uso correto dos métodos HTTP
- Respostas padronizadas

Exemplo prático (contexto de conta bancária)

Criar conta (POST)

`POST /contas`

Body:

```
{
  "nome": "João",
  "saldo": 1000.00
}
```

Buscar conta (GET)

GET /contas/1

Transferir valor

POST /contas/transferencias

Body:

```
{
  "origemId": 1,
  "destinoId": 2,
  "valor": 100.00
}
```

Códigos de status HTTP

A API também responde com códigos:

Código	Significado
200	OK
201	Criado
400	Erro na requisição
404	Não encontrado
500	Erro no servidor

Tecnologias comuns

Para construir APIs REST em Java:

- Spring Boot
- JAX-RS

Outras tecnologias:

- Node.js
- Django REST Framework

Boas práticas

- Usar nomes de recursos no plural (`/clientes`)
- Evitar verbos na URL (`/criarCliente` ❌ → `/clientes` ✅)
- Utilizar corretamente os métodos HTTP
- Versionar API (`/v1/clientes`)
- Tratar erros corretamente

Vantagens

- Simples de entender e usar
- Escalável
- Independente de tecnologia
- Muito usado em sistemas modernos (web e mobile)

Resgatar oferta

Fale sobre api rest